Abstract

This paper describes a small, tested **domain language for specifying controlled methods** — the **methods-paper exemplar** of the Research Project Template. Unlike a results paper, this manuscript's subject is the methodology itself: a controlled vocabulary, a unit system with dimensional safety, four staged validation gates, and a deterministic compiler, implemented in `projects/templates/template_methods_paper/src/template_methods_paper/methods_dsl/` and described section by section in sec. 3. The domain language's vocabulary is informed by BPL (Biology Programming Language, (Bota Biosciences 2026)), an upstream reference that encodes laboratory protocols as programs with biology-native types, staged validation, and deterministic compilation; this exemplar generalizes BPL's intent vocabulary and pipeline shape from wet-lab protocols to any

controlled procedure.

A `Method` is a name, a set of typed parameters and resources, and an ordered, dependent set of steps — constructed directly as frozen Python dataclasses (`src/template_methods_paper/methods_dsl/model.py`) rather than parsed from new text syntax. Every `Quantity` carries a unit that resolves to one of 18 controlled units across eight dimensions, and every step names one of 9 controlled-vocabulary intents (`src/template_methods_paper/methods_dsl/vocabulary.py`), executable on one of 3 backends. 4 staged gates — structural, semantic, plan, and target — validate a method before `compile_method` (`src/template_methods_paper/methods_dsl/compiler.py`) deterministically schedules it with Kahn's algorithm (Kahn 1962) and hashes the canonical plan with SHA-256.

Abstract I

We demonstrate the language on 2 worked example methods spanning both domains BPL's design targets and the domains it generalizes to: a manual wet-lab preparation (`PBSPreparation`, 5 steps, target `human`, plan hash `313b9b17de98`) and an automated instrument-calibration procedure (`SensorCalibrationSweep`, 4 steps, target `automated`, plan hash `d89cced19be6`). Live re-compilation determinism check: **Yes**. Across both methods, 8 of 8 staged-gate evaluations pass. A demonstration provenance hash-chain (`src/template_methods_paper/methods_dsl/trust.py`) of length 3 verifies as **Yes**.

Contributions are **methodological** and **architectural**. On the methods side, we show that a controlled vocabulary expressed as typed dataclasses — not a parsed grammar — is sufficient to reproduce BPL's core safety properties (dimensional safety, staged validation, deterministic compilation) at a scope appropriate for a template exemplar. On the architecture side, the DSL is exercised by a zero-mock test suite under the repository's configured project coverage gate, generates 19 artifacts (1 figures, 7 data files, 11 reports) per pipeline run, and injects reproducibility metadata (configuration hash `0937b126ad9a5a71`, build timestamp `2026-10-31T05:06:21Z`) into sec. 7.

Abstract I

Keywords: methods paper, domain-specific language, controlled methods, deterministic compilation, staged validation, dimensional analysis